Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Analysis of financial markets using machine learning

Bachelor Thesis

2026

Marek Šugár

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Analysis of financial markets using machine learning

Bachelor Thesis

Study Programme:	Data Science
Field of Study:	Computer Science and Mathematics
Department:	Department of Applied Mathematics and Statistics
Supervisor:	Mgr. Samuel Rosa, PhD.

Bratislava, 2026

Marek Šugár

17424822

Comenius University Bratislava
Faculty of Mathematics, Physics and Informatics

THESIS ASSIGNMENT

Name and Surname:	Marek Šugár
Study programme:	Data Science (Joint degree study, bachelor I. deg., full time form)
Field of Study:	Computer Science Mathematics
Type of Thesis:	Bachelor´s thesis
Language of Thesis:	English
Secondary language:	Slovak

Title: Analysis of financial markets using machine learning

Annotation: Modeling and prediction of stocks in financial markets have long been a researched problem. Historically, this problem was approached using specific methods of financial mathematics, but with the significant development and success of machine learning in general, machine learning methods have also begun to be applied to predicting the behavior of financial markets. The aim of this thesis is to create a prediction model for selected stocks, which involves the appropriate selection and identification of relevant explanatory variables, as well as the suitable choice of model. Subsequently, it will be possible to analyze whether the obtained predictions can be used for successful creation of an investment portfolio.

Supervisor:	Mgr. Samuel Rosa, PhD.
Department:	FMFI.KAMŠ - Department of Applied Mathematics and Statistics
Head of department:	doc. Mgr. Igor Melicherčík, PhD.
Assigned:	02.10.2025
Approved:	28.10.2025

doc. Mgr. Tomáš Vinař, PhD.
Guarantor of Study Programme

.....
Student

.....
Supervisor

Declaration: I hereby declare that I have independently written this entire bachelor thesis entitled "Analysis of financial markets using machine learning", including all its figures and appendices, using the literature listed in the attached bibliography.

In preparation of the thesis I have also used the LLM ChatGPT for the keyword-based searches in particular Python libraries' documentations and thesis' grammar checks. I have used artificial intelligence tools in accordance with relevant legal regulations, academic rights and freedoms, ethical and moral principles while simultaneously maintaining academic integrity. I am aware that I am fully responsible for the accuracy of the final text.

Acknowledgments: This way I would like to thank my supervisor Dr. Rosa, for his mentorship, spent time and intriguing feedback throughout the whole process of thesis' creation. Last but not least, I would like to express thanks to all my family for their endless support and motivation.

Abstrakt

Na tomto mieste sa bude nachádzať slovenský abstrakt

Kľúčové slová: strojové učenie, akciové trhy, prediktívna analýza

Abstract

Here will be English written abstract.

Keywords: machine learning, stock markets, predictive analysis

Contents

Introduction **1**

1 Machine learning foundations **2**

1.1 Machine learning as a method 2
1.2 Bias-variance trade-off 3
1.3 Linear regression 4
1.4 Regularized variants of linear regression 6
1.5 k-Nearest neighbours algorithm 7
1.6 Decision tree regression 7
1.7 Time-series analysis (ARIMA) 9

2 Stock market data **11**

2.1 Variables in financial data 12
2.2 Rolling window and prediction methodology 13
2.3 Suitable features in modeling 15
2.3.1 Stock indices 15
2.3.2 Commodities 16
2.3.3 Currency pairs - FOREX 17
2.3.4 Other market indicators 17

3 Predictive analyses on IT segment **19**

3.1 Evaluation metrics 19
3.2 Linear regression 20
3.2.1 Possible feature selection 20
3.2.2 Optimization of p 21
3.3 Lasso regularization 23
3.4 Ridge regularization 26
3.5 k-Nearest neighbours regression 28
3.6 Random forest regression 31
3.7 ARIMA regression 32
3.8 Test evaluation 33

4 Next-day optimization of portfolio based on model's prediction **35**
4.1 Markowitz mean-variance portfolio 35
4.2 Portfolio evaluation metrics 35
4.3 Evaluation on the training-validation interval 35
4.4 Evaluation on the test interval 35

Conclusion **36**

References **37**

Appendix A **40**

List of Figures

1.1 Balancing the total error of the model through bias and variance of it. . 4
1.2 Illustrative example of residuals in the context of linear regression. . . . 5

2.1 Visualisation of the defined attribute's movement terms on Microsoft stock price. 13
2.2 Sample algorithm step for predicting the closing price of a given stock on January 20, 2022, with $p = 4$. 15
2.3 VIX value movement through the 2025. 18

3.1 MAPE dependence on number of features selected (stock ENPH). . . . 21
3.2 MAPE dependence on the rolling window training length parameter (stock ENPH). 21
3.3 Errors in prediction for single stocks (linear regression). 22
3.4 MAPE dependence on the rolling window training length parameter and regularization parameter (stock ENPH). 23
3.5 MAPE dependence on the rolling window training length parameter and regularization parameter in Lasso (stock ENPH). 24
3.6 Errors in prediction for single stocks (General model Lasso). 25
3.7 MAPE dependence on the rolling window training length parameter and regularization parameter in Ridge (stock ENPH). 26
3.8 MAPE dependence on the rolling window training length parameter and regularization parameter in Ridge (stock ENPH). 27
3.9 Errors in prediction for single stocks (General model Ridge). 28
3.10 Dependence of k and p parameters on MAPE of next-day predictions (k-NN; average among chosen stocks). 29
3.11 Errors in prediction for single stocks (k-NN). 30
3.12 Dependence of MAPE on the length of the training window p (stock ENPH, Random Forest). 31
3.13 Dependence of model dominance on training window length (stock ENPH). 32

List of Tables

3.1 MAPE and dominance values for different settings of the model (Lasso). 24
3.2 MAPE and dominance values for different settings of the model (Ridge). 27
3.3 Evaluation of tuned models on the training-validation interval. 33
3.4 Evaluation of tuned models on the test interval. 34

Introduction

"In this business, it's easy to confuse luck with brains."
— Jim Simons (founder of Renaissance Technologies)

Here will be the Introduction chapter, where reader will be introduced to the topic of Quantitative Science, its novelty and to the topics being dealt with in academic community and practice as well.

Chapter 1

Machine learning foundations

The aim of this chapter is to overview the methods of machine learning, which are going to be used for the purposes of predictive regression modeling in this thesis. The chapter starts with a brief overview of machine learning as a method, followed by descriptions of the individual algorithms used.

1.1 Machine learning as a method

Nowadays, we can say that machine learning (ML) has become a widely known term in the field of data science. It stands for the subset of artificial intelligence enabled to identify patterns in data [2]. Generally speaking, ML implements algorithms capable of learning from data and generalizing patterns to be able to make predictions on unseen data, which means being able to perform tasks without being explicitly told to [9].

Mathematically speaking, ML builds on the idea of having a quantitative target variable Y and p various predictors $X_1, X_2, \ldots, X_p$. We assume the existence of an underlying function f such that

$$Y = f(X_1, X_2, \ldots, X_p) + \epsilon, \tag{1.1}$$

where ϵ is a random noise independent of $X_1, X_2, \ldots, X_p$. Algorithms and methods of ML aim to estimate the function f by using a so-called **training dataset** of observed data.

Suppose we have n observations of the introduced predictors and the corresponding target Y as $\{Y_i, X_{i,1}, X_{i,2}, \ldots, X_{i,p}\}_{i=1}^{n}$, where $X_{i,q}$ denotes value of the q-th predictor in the i-th observation. Then, ML algorithms use this dataset to approximate function $\hat{f}$, such that $\hat{f}(X_{i,1}, X_{i,2}, \ldots, X_{i,p}) = \hat{Y}_i \approx Y_i$, for all $i = 1, 2, \ldots, n$. With the obtained $\hat{f}$, we can make predictions of Y for new (unseen before) values of predictors $X_1, X_2, \ldots, X_p$.

Based on the methodology used to find $\hat{f}$, we can distinguish two methods – parametric and non-parametric. **Parametric methods** seek to approximate f by specifically optimizing a various number of parameters, which then explicitly define the function $\hat{f}$. This involves a prior assumption about the functional form of f (e.g., linear, polynomial, etc.). Then, using specific algorithms, the parameters of the function are estimated from the training data, resulting in $\hat{f}$. Examples of such methods used in this thesis are linear regression or its regularized variants.

On the other hand, **non-parametric methods** do not assume any specific functional form of f. Instead, they rely on patterns included in the data itself to roughly approximate f. Generalisation to unseen data is achieved by identifying patterns and similarities in the training data, without explicitly defining $\hat{f}$. An example of such a method used in this thesis is the k-nearest neighbours algorithm or the random forest algorithm.

Lastly, based on the character of the Y, we can distinguish regression and classification tasks. In **regression** tasks, Y is a continuous variable, while in **classification** tasks, Y is a categorical variable.

1.2 Bias-variance trade-off

Lastly, in order to provide a complete overview of the theoretical foundations of ML, it is important to mention the concepts of overfitting and underfitting [20]. The key concepts involved in this are **bias** and **variance**.

Bias is the error arising from incorrect assumptions made by the model. One of the many examples is assuming that f belongs to a certain functional class; however, the actual type is much more complex (e.g., restricting f to be linear instead of polynomial). High bias causes the model to be insufficiently complex to generalize patterns in the training dataset, meaning it cannot perform well on previously unseen data. This is called **underfitting**.

Variance is the error arising from the model's sensitivity to perturbations made in the training dataset. The problem arises when a particular training set yields trained model, while another, slightly perturbed dataset, yields marginally different model, which usually leads to unstable generalization performance. This is mainly caused when the complexity of the model is too large (e.g., a high-order polynomial). In such a situation, the model memorizes patterns from the training data rather than learning the underlying patterns. This phenomenon is called **overfitting**.

In practice, the goal is to choose the complexity (e.g., by assuming a functional type or by selecting a particular model) to achieve lowest possible bias and variance simultaneously (see Figure 1.1).

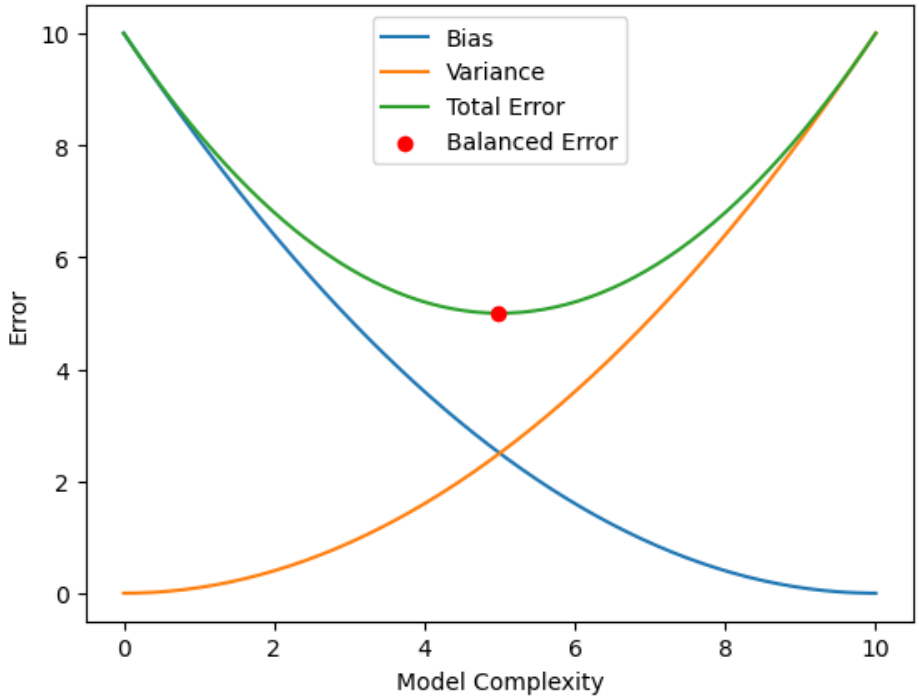

Figure 1.1: Balancing the total error of the model through bias and variance of it.

1.3 Linear regression

Linear regression is one of the most fundamental methods in ML. It is a parametric method used for regression tasks, assuming that the true relationship between the predictors and the target variable is linear [9].

Given a training dataset of n observations $\{Y_i, X_{i,1}, X_{i,2}, \ldots, X_{i,p}\}_{i=1}^{n}$, we can formulate the linear relationship as

$$Y_i = \beta_0 + \beta_1 X_{i,1} + \ldots + \beta_p X_{i,p} + \epsilon_i, \qquad i = 1, 2, \ldots, n \tag{1.2}$$

with ϵ_i representing an independent noise. Formulating these equations to a more convenient matrix notation, we obtain

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}, \tag{1.3}$$

where

$$\mathbf{Y} = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} 1 & X_{1,1} & \ldots & X_{1,p} \\ 1 & X_{2,1} & \ldots & X_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & X_{n,1} & \ldots & X_{n,p} \end{bmatrix} \qquad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix} \qquad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

In order to derive one of the methods to approximate the parameters $\boldsymbol{\beta}$, it is necessary to define **residuals**. A residual represents the deviation of the obtained approximation by the model from the true value of the target (Figure 1.2). When the model, defined by certain $\hat{\beta}$, approximates Y as $\hat{Y}$, then $Y - \hat{Y}$ is the residual.

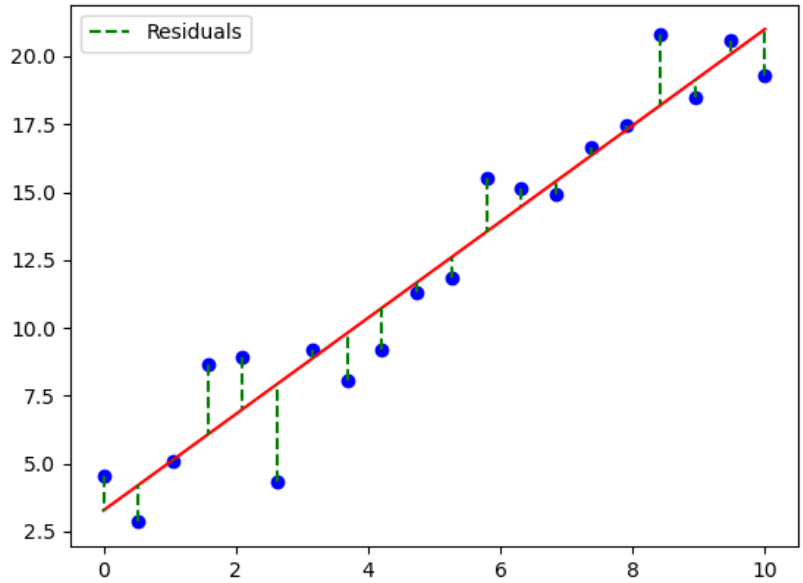

Figure 1.2: Illustrative example of residuals in the context of linear regression.

Using the residuals, we can measure the goodness of fit. We aim to define the linear relation to be as close as possible to the training data. In matrix notation, we can represent the residuals for all n available observations as $Y - \mathbf{X}\hat{\boldsymbol{\beta}}$. To ensure that large errors are penalised more heavily and all of the residuals contribute positively, regardless of their sign, the general error function is defined as

$$(\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})^{\mathrm{T}}(\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = \|\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|_2^2 \tag{1.4}$$

or in an equivalent way as sum of squared residuals as

$$\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

where $\hat{Y}_i$ denotes the model's approximation of Y_i.

Based on the Equation (1.4), we can derive an optimization approach to derive $\hat{\beta} \approx \beta$ as

$$\arg\min_{\hat{\beta}} \|\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|_2^2 \tag{1.5}$$

The approach in Equation (1.5) stands for **least-squares estimation**. Other estimation methods for $\hat{\beta}$ exist; however, only the least-squares approach is included in the chapter, as it is used in the subsequent computational implementation.

Apart from the approximation methods, it is possible to derive closed form of the solution to the Equation (1.5) [20]. Straightforwardly, the value of the derivative with respect to the $\hat{\boldsymbol{\beta}}$ of the Equation (1.4) is set to zero as

$$\frac{\partial}{\partial \hat{\boldsymbol{\beta}}} \|\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|_2^2 = -2\mathbf{X}^{\mathrm{T}}(\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = 0$$

then, solving for $\hat{\boldsymbol{\beta}}$ yields

$$\mathbf{X}^{\mathrm{T}}\mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}^{\mathrm{T}}\mathbf{Y} \Rightarrow \hat{\boldsymbol{\beta}} = (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{Y}. \tag{1.6}$$

Such solution arises from the convexity of the Equation (1.4).

1.4 Regularized variants of linear regression

Linear regression might overfit certain types of datasets. This problem arises with large values of the parameters in $\boldsymbol{\beta}$. Relatively large values of the parameters result in significant changes in predictions $\hat{Y}$, even under slight input perturbations, thus making the predictions potentially imprecise. There exist a plenty of methods involving explicit feature selection, resulting in omitting the features with large $\hat{\beta}$; however, by using **regularized methods**, the approach of feature selection could be avoided [9, 20].

Continue with Equation (1.5) and let us add a specific term taking into account the sum of absolute values of the single parameters $\hat{\beta}$

$$\arg\min_{\hat{\beta}} (\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})^{\mathrm{T}}(\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}) + \lambda\|\hat{\boldsymbol{\beta}}\|_1, \tag{1.7}$$

where $\lambda \geq 0$ is a so-called tuning parameter and $\|\hat{\boldsymbol{\beta}}\|_1 = \sum_{i=1}^{p} |\hat{\boldsymbol{\beta}}_{\boldsymbol{i}}|$ is a L1 norm. Equation (1.7) trades off two criteria – on the one hand, equally as in linear regression, the model seeks $\hat{\boldsymbol{\beta}}$ which fits the data the best, but on the other hand, the model seeks a fit with the smallest possible sum of absolute values of the elements in $\hat{\boldsymbol{\beta}}$. This method is called **Lasso regularization** of linear regression and fully aligns with the objectives of explicitly preserving all the features, while limiting the possibility of overfitting by minimizing the parameters in $\hat{\boldsymbol{\beta}}$.

However, Lasso regularization, while optimizing, might set some of the parameters exactly to zero, which results in an implicit **feature selection** process and the elimination of potentially invaluable features [9].

Building up on the method introduced by Equation (1.7), it is possible to use another type of norm to penalize values of the parameters in $\hat{\boldsymbol{\beta}}$.

$$\arg\min_{\hat{\beta}} (\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})^{\mathrm{T}}(\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}) + \lambda\|\hat{\boldsymbol{\beta}}\|_2^2, \tag{1.8}$$

where $\lambda \geq 0$ is again a tuning parameter and $\|\hat{\boldsymbol{\beta}}\|_2^2 = \sum_{i=1}^{p} \hat{\boldsymbol{\beta}}_i^2$ is a L2 norm. The approach in Equation (1.8) is then called **Ridge regularization** of linear regression. The goal of this optimization problem aligns with that in Lasso, preserving the trade-off between fit and minimal parameters in $\hat{\boldsymbol{\beta}}$.

Apart from the Lasso, Ridge generally does not make the parameters equal to zero, meaning it does not implicitly perform feature selection [9].

The most important property of these methods, along with the parameters minimization, is the model variance decrease. With a suitably chosen λ, along with the variance decrease, the bias tends to increase, which typically results in better performance on unseen data – better generalization of the model [9].

1.5 k-Nearest neighbours algorithm

The main idea, which forms the foundation of this method, is based on the assumption that feature data points residing in the given vector space, which are close to each other (for some distance definition), tend to be more similar in terms of the target variable Y [20]. This method can solve both regression and classification tasks; however, only the regression setup is explained, as only this is used in the thesis.

Suppose we have the training dataset as earlier, $\{Y_i, \boldsymbol{X}_i\}_{i=1}^n$, where the stacked vector $\boldsymbol{X}_i = (X_{i,1}, X_{i,2}, \ldots, X_{i,p})^T$. Further, we have a new datapoint with vector of predictors $\hat{\boldsymbol{X}}$, but without the linked value of the target Y. Lastly, we define a distance metric $\|.\|$ and a parameter $k \leq n$.

Let $N \subseteq \{i\}_{i=1}^n$ be the set of indices of the k nearest training feature vectors $\boldsymbol{X}_i$ to the $\hat{\boldsymbol{X}}$, in terms of $\|.\|$ distance. Then, the approximation of the target Y is

$$Y \approx \frac{1}{k} \sum_{i \in N} Y_i \tag{1.9}$$

This method is non-parametric and aims to uncover patterns solely using the given training dataset. This method is called the **k-nearest neighbours** algorithm (k-NN).

1.6 Decision tree regression

The idea of splitting the vector space of data into subsets based on occurrence of individual data points in training dataset, as in the k-NN, is particularly easy to interpret and replicate. However, the problem might arise with uniform weighting of all predictors, as there might be predictors with larger or lower importance.

To preserve the high interpretability of space partitioning and allowing for weighting of individual predictors, data structure **binary tree** comes as a very natural candidate. The idea of using a tree is to recursively partition training dataset into tree's nodes,

representing certain space partition, and effectively terminate the tree at certain depth, with leaf nodes whose data points define the resulting target approximation.

Similarly to the k-NN case, this algorithm is capable of solving both regression and classification tasks; however, only the regression setup in explained.

Let $\{Y_i, X_{i,1}, X_{i,2}, \ldots, X_{i,p}\}_{i=1}^n$ be the training dataset. The complete dataset is associated with the root node of the tree structure. The tree-building algorithm aims to recursively divide this set into two disjoint subsets based on a decision criterion $X_j \leq w$ and $X_j > w$, where X_j is the certain chosen predictor among $\{X_q\}_{q=1}^p$ and $w \in \mathbb{R}$ specified threshold value.

The building of the tree is a greedy algorithm, which means that, at each step of the building, the algorithm chooses partition criterion locally optimally according to the current set of datapoints.

Let S be the set of datapoints in a given node. The aim is to define S_{left}, S_{right} belonging to left and right child nodes respectively, based on the division criterion. In regression, the goal is to divide S, to make S_{left}, S_{right} *as homogeneous as possible.* We can define the weighted average of Y variances, in respective sets as

$$\text{Split}(X_j, w) = \frac{|S_{left}|}{|S|}\text{Var}(S_{left}) + \frac{|S_{right}|}{|S|}\text{Var}(S_{right}) \tag{1.10}$$

where $|S_*|$ denotes the cardinality of the set S_* and $Var(S_*)$ denotes variance of the target values within S_*. This metric is explicitly defined by S and the chosen X_j, w. With this definition, we can measure homogeneity of the resulting split, making variance of both S_{left} and S_{right} as low as possible.

This means that in all nodes of the structure, the algorithm solves

$$\arg\min_{X_j, w} \text{Split}(X_j, w) \tag{1.11}$$

and performs the split until a termination criterion is satisfied (e.g., maximum depth of the tree, minimum points in leaves, etc.).

With such a data structure built, a certain input $\hat{\boldsymbol{X}}$ belongs to particular leaf node. The approximation of Y corresponding to the $\hat{\boldsymbol{X}}$ is then the arithmetic mean of the leaf's training Y values. This approximation method is called the **decision tree algorithm** [9].

Building an approximation solely on one decision tree might result in overfitting. Since a single tree is built using greedy approach, even small perturbation of the training dataset might potentially result in substantially different tree structures and, consequently, very different predictions.

This sensitivity arises from the locally optimal splits in nodes, which depend heavily on the sample of data used in training. As a result, trees tend to overfit the data, particularly at relatively large tree depths [9, 20].

A possible solution comes from the statistical method of **bootstrap** – artificial samples are generated from the original data by sampling with repetition [20]. We define B as the number of such bootstrap samples, and single decision trees are trained on these samples. Then, each tree outputs a target approximation for the input as $\hat{Y}^{(b)}$. Then, the final approximation $\hat{Y}$ is

$$\hat{Y} = \frac{1}{B} \sum_{b=1}^{B} \hat{Y}^{(b)}.$$

This resampling strategy introduces diversity among individual trees, which is essential for reducing the variance in the final model [9, 20].

1.7 Time-series analysis (ARIMA)

In all of the models introduced earlier, the idea of training the model was to tune the model to be able to generalize the input variables into output, the target variable. For the purposes of predicting the future developments of time series, there exist specialized approaches beyond the mentioned conventional ML frameworks [4]. One of the most widely used frameworks is the **Autoregressive Integrated Moving Average** (ARIMA) model.

The ARIMA model relies on capturing short-term and often temporal patterns in the time series. Apart from the ML algorithms explained earlier, ARIMA is time-aware, meaning its framework directly works with the concept of time flow and the dependency of target variable on it [4].

Let there be a univariate time series of length T as $\{Y_t\}_{t=1}^{T}$. The idea of the ARIMA model relies on the concept of autoregressivity, which means that future data points of the time series are dependent on the earlier points in series. Rigorously speaking, let there be $p \in \mathbb{N}_0$ and

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \ldots + \phi_p Y_{t-p} + \epsilon_t \tag{1.12}$$

where ϵ_t is the irreducible noise term and $\phi_1, \phi_2, \ldots, \phi_p$ are **autoregressive coefficients**. Time series tend to be non-stationary, which means that their statistical properties tend to change over time. For this purpose, let there be an operator B, which satisfies $BY_t = Y_{t-1}$. Then, the term of **differencing** of the d-th order is defined as

$$\Delta^d Y_t = (1 - B)^d Y_t. \tag{1.13}$$

Differencing removes the trend of the time series, stabilizes it, and decreases its non-stationarity.

Lastly, the idea of ARIMA relies on the property of the **moving average**, which means that ARIMA models the value of Y_t as a linear combination of forecast errors made in the past:

$$Y_t = \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \ldots + \theta_q \epsilon_{t-q} \tag{1.14}$$

where $\theta_1, \theta_2, \ldots, \theta_q$ are **moving average coefficients**. ARIMA tends to incorporate deviations encountered in the past to improve future predictions.

By combining properties introduced in Equations (1.12), (1.13) and (1.14), and using the chosen parameters p, d, q, we obtain the complete model defined as ARIMA$_{(p,d,q)}$ [4].

$$\Delta^d Y_t = \phi_1 \Delta^d Y_{t-1} + \ldots + \phi_p \Delta^d Y_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \ldots + \theta_q \epsilon_{t-q} \tag{1.15}$$

The model relies on the assumption that $\epsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$. Under this assumption, the joint likelihood of the observed time series can be written as

$$\mathcal{L}(\boldsymbol{\phi}, \boldsymbol{\theta}, \sigma^2 | Y_1, Y_2, \ldots, Y_T) \tag{1.16}$$

where $\boldsymbol{\phi} = (\phi_1, \phi_2, \ldots, \phi_p)^T$ and $\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_q)^T$. These parameters are estimated by solving

$$\arg\max_{\boldsymbol{\phi}, \boldsymbol{\theta}, \sigma^2} \mathcal{L}(\boldsymbol{\phi}, \boldsymbol{\theta}, \sigma^2)$$

Such a model is then capable of predicting the future development of the time series variable, solely using its past development.

Chapter 2

Stock market data

The aim of this chapter is to introduce a set of variables related to the financial market, their possible importance in terms of predictive modelling, and lastly their subsequent processing into specific forms, which were used throughout all the related analyses in this thesis.

Market itself and particularly the price movement of financial instruments is a complex and unsurprisingly counterintuitive process with a large number of various related variables, which contribute to price movements. In terms of using such data as an auxiliary source for decision making in investment strategy creation, there is a rich universe of sources dealing with the issue [7, 14].

In the thesis, we aim mainly to research the predictive power of various diversely sophisticated methods of machine learning on the particular market segment. Many papers tend to focus on a single stock or a very limited set of a few stocks [8, 24]. Others tend to apply predictive modelling methods to stock indices (e.g. US S&P 500 and NASDAQ Composite or German DAX) [18] however, there is a small number of sources dealing with a market segment, analyzing single stocks included in it.

Our main focus is on the index S&P 500 – a US stock index accumulating the 500 biggest publicly traded companies in terms of market capitalization. The index itself is considered to be the most trustworthy general indicator of the overall US and world's stock economy state [10]. The index itself is divided into several general market segments to which stocks belong. In this thesis, we focused on the segment **Information technology**. However, the approach used in our analyses is adaptable to any subset of stocks – the only determining constraint is the respective database of stocks.

As of late 2025, the Information technology (IT) segment consists of 68 US-traded stocks. However, we excluded three stocks, as their brief period of trading yielded insufficient historical data for reliable modeling. In the year 2025, in terms of total capitalization, the IT group makes a total of nearly 30 % of the index, which makes

it the most weighted sector, despite making only 13.6 % of all included stocks [16]. This recent growth of its weight could be explained by a lot of factors steering the market nowadays, but the general view on the topic suggests that recent boom of artificial intelligence could be the most dominant engine of growth. This suggests that focusing on this segment is reasonable, as it could be considered the most important one nowadays.

2.1 Variables in financial data

At first, we introduce specific attributes which define the value movement of all financial and economic instruments used throughout the thesis as features and target variables. These attributes could be further generalized onto any financial instrument attribute's movement, not only to price itself – as it will be applied to non-stock data as well.

For the purposes of defining desired terminology, let's define a time interval on which we analyse an instrument's attribute movement as $[t_1, t_2]$, where naturally $t_1 < t_2$. Defined attributes, which are attached to a single time breakpoints, could be universally defined for any time interval with various frequency.

At the breakpoint t_1 we define the **opening value**, which denotes the value of the attribute at the very beginning of the interval. For the purposes of this thesis, where we aim to use a daily frequency of data, this value denotes the value of the attribute at the very start of the trading day. In the opposite manner, at the breakpoint t_2 we define the **closing value**, denoting the final value of the attribute at the end of the observed interval – in our case, at the end of the trading day.

During the process of value evolving throughout the interval, we observe the global minimum and maximum of the attribute's value, which we denote **low value** and **high value**, respectively. Such data give us an indication of volatility during the interval and subsequently the sentiment of the market regarding the attribute.

The visualisation of the terms for a sample interval of a Microsoft stock between the beginning of June 2025 and the beginning of September 2025 is pictured in Figure 2.1.

Last but not least, we accumulate data regarding the number of an instrument's stocks traded (either sold or bought) during the time interval. This is denoted as **volume**, and in terms of the thesis, the value of this attribute is known for stocks only; for other instruments we use the value of zero.

The last data attribute, applied only on stocks' data as well, is volume of **dividends** paid out. Certain companies oblige themselves to pay parts of their profit to their investors on a regular basis (e.g., Apple, Microsoft or Oracle) [3]. Some companies do not pay out dividends, even though this attribute was included in the database for all

included stocks.

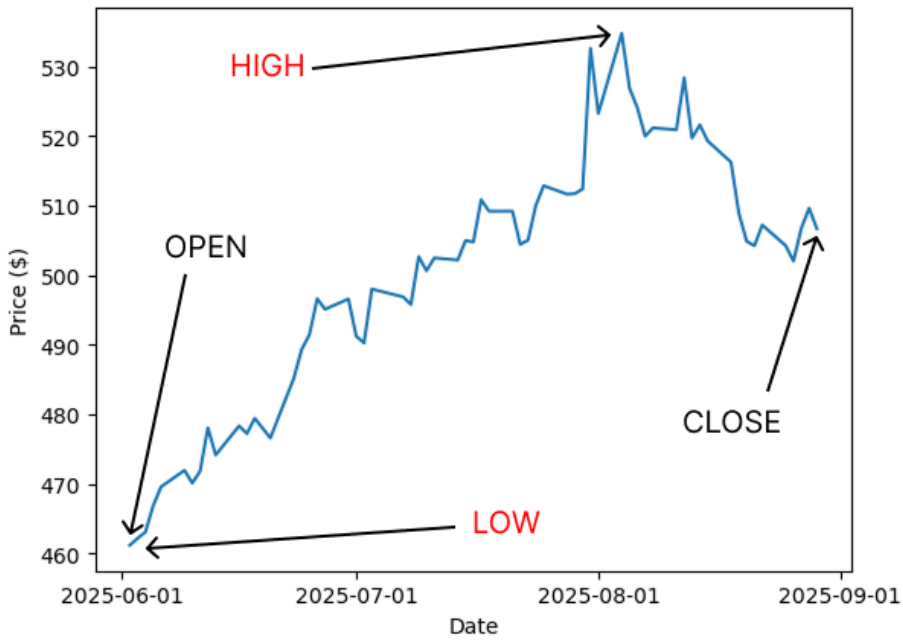

Figure 2.1: Visualisation of the defined attribute's movement terms on Microsoft stock price.

2.2 Rolling window and prediction methodology

There are several approaches for splitting time-series data into training, validation, and test sets. For the purposes of next-day prediction — where data from trading days $t-q, t-(q-1), \ldots, t-1$ (for some q) are used to predict the closing price on day t — the concepts of rolling window methodology are adopted [4].

In our approach, we define a static time-series interval, which serves as a hybrid train-validation time interval. Let's define the overall data interval being between T_1 and T_2 timepoints (let $T_2 - T_1 = M$). On this interval reside two frames of variables.

Multivariate $\mathbf{X} \in \mathbb{R}^{M \times F}$, where F denotes the number of features used, which serves as an input variable. Included features in $\mathbf{X}$ are values of selected financial variables, shifted backwards up to $q = 9$ days. This means, that $\mathbf{X}[t]$ (the t-th row of $\mathbf{X}$) represents data available through days $t-1, t-2, \ldots, t-9$. [1]

Second, univariate $Y \in \mathbb{R}^M$ accumulates values of the target variable (next closing price) to the respective rows of $\mathbf{X}$. This means, that $Y[t]$ (the t-th value in the Y) is

[1] We chose the value of q due to few reasons constant. Firstly, variable q would result in larger time-complexity and necessity to optimize new parameter further. Secondly, we want to emphasize short-term market dynamics, which are commonly observed in financial time series and documented in the literature [4].

the value of the closing price on day t.

Further, we define parameter $1 \leq p < M$, which is the length of the training subwindow, meaning how many *next-day* predictions model would be trained on, and w, which denotes the forecasting horizon, i.e., the number of time units into the future for which predictions are made. For the purposes of this thesis and its next-day prediction methodology we define $w = 1$.

With respect to the adopted methodology, we can generalize the process of train and validation sets split, by creating a pseudocode demonstrating the relationship between input and target matrices. Algorithm 1 shows what parts of the global $\mathbf{X}, Y$ are directly available at the certain moment of model building.

Algorithm 1 Rolling-window on (T_1, T_2) interval

```
# We assume w ← 1
# The prediction target day is t + p + 1, we have available data up to day t + p
Set look-back window length p
Retrieve the feature variables' values into the X
for each stock in database do
    Retrieve closing prices into the target Y
    for t = T_1 to T_2 − (p + 1) do
        X_train ← X[t : t + p]
        Y_train ← Y[t : t + p]
        X_predict ← X[t + p + 1]
        # X_predict corresponds to the latest available data
        Train model f̂ on (X_train, Y_train): so Ŷ_train = f̂(X_train)
        Generate prediction: f̂(X_predict)
    end for
end for
```

From Algorithm 1 a few important properties of this method could be derived. Firstly, respective intervals are overlapping and the shift between respective input and target matrices, at t and $t + 1$, is a single time unit. Secondly, by allowing window overlapping, we maximize the number of possible training data points, making the training-validation process possibly more robust and general. Lastly, such setup would be implemented on train-validation interval (T_1, T_2) and on disjoint test interval likewise.

The multivariate $\mathbf{X}$ contains pre-chosen shifted variables (Subchapter 2.3). In this thesis, time interval between September 15, 2017 and December 31, 2022 (1333 trading days) is used as training-validation interval and time interval between January 1, 2023 and January 16, 2026 (761 trading days) is used as test interval.

At this point, it is sufficient to illustrate this data preparatory setup with a specific example in order to better understand Algorithm 1. Let $p = 4$ and suppose we want to predict the closing price on January 20, 2022. We have complete data up to January 19, 2022. With fixed $q = 9$ and given p the available training and validation data points are depicted in Figure 2.2.

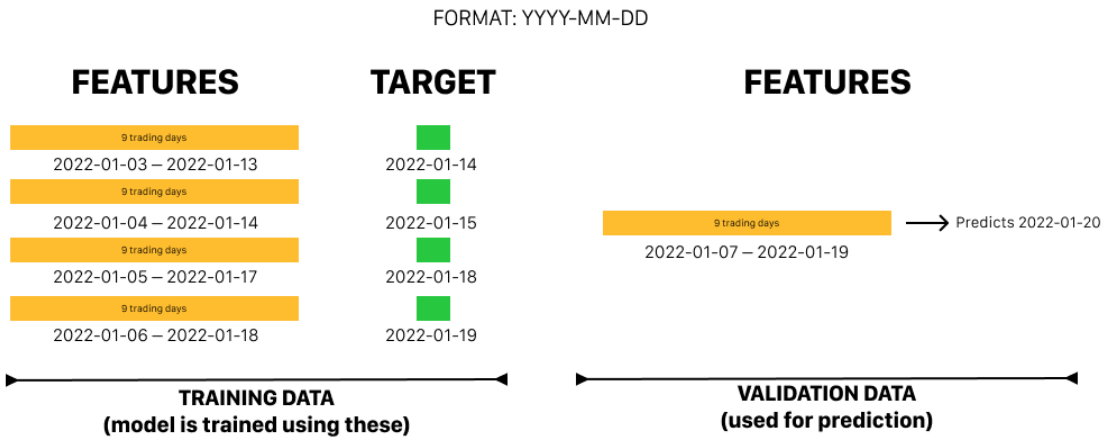

Figure 2.2: Sample algorithm step for predicting the closing price of a given stock on January 20, 2022, with $p = 4$.

Such setup is iteratively being done for every single stock, both on train-validation and test intervals.

2.3 Suitable features in modeling

The amount of variables, which are directly or indirectly generated together with day-to-day market running, is quite enormous, and one should consider subsetting them for machine learning purposes wisely. In this subchapter, we aim to show and justify the process of our general feature selection, mostly based on a priori knowledge of processes within financial markets and the variables' most likely effect on stock markets.

For the sake of completeness, data were obtained from the API directly fetching data available on Yahoo Finance – available through the Python library abstraction provided by the `yfinance` library [1, 23].

2.3.1 Stock indices

In order to process the current general state of the stock exchange we researched possibilities of implementing data regarding the price development of certain stock indices. **Index** is an abstract concept of accumulating information regarding single

stock assets into one unified asset, mostly using a predefined weighted average of prices of the stocks included.

The stock index is specifically designed to follow and replicate the performance of a given subset of stocks. Taking this into account, by including such well-chosen indices, which copy a broader and more various scope of stocks on a given market, we may obtain valuable information regarding the current state of the US market.

For these purposes, we decided to include into our database two major stock indices. The general index **Standard's and Poor 500**, commonly known as S&P 500, is an index including the 500 biggest US-traded stocks based on total market capitalization. The index consists of a wide scope of various segments making it robust and universal in terms of mirroring the overall US market state.

Taking into account the scope of this thesis – the Information technology segment, we decided to include the index **NASDAQ Composite** (NASDAQ). The main difference from S&P 500 is its segment focus. NASDAQ focuses mainly on tech and IT companies, making it possibly a valuable piece of data for our database of feature data.

2.3.2 Commodities

Commodities denote mainly raw resources and materials, which are traded mainly in uniform quality and large quantities. Their fundamental role in the market comes from the fact that most of these assets play an essential part in further processing, consumption and industry. Matsumoto et al. suggest, that the commodity market play a vital part in the whole financial segment, which suggests, that including such assets in the database might be sufficient [15].

For the purposes of this thesis, we included data regarding two major commodities – **crude oil** and **gold**. Even though there might not be a clear connection to the IT sector, we believe that such data would bring valuable information about the general state of the whole global economy.

Oil plays a vital role in keeping the world in motion, and a solid majority of the world's activities depend on its presence and availability. This might imply that these dependencies on the global market would be projected into the price movement of the commodity.

Last but not least, we introduce gold price movements into the database. For a wide range of reasons, gold still remains one of the most prominent securities. Most modern currencies are dependent on a system, where gold acts as a security stabilizing currency movements and limiting the governments issuing money. On top of that, governments rely on physical gold as a reserve in times of crises [17]. Taking these reasons into account, by including gold in the database of features, we assume we are obtaining valuable information regarding the world's economy state, crisis potential or

investors' sentiment, etc.

2.3.3 Currency pairs - FOREX

Currency pairs and subsequently the **Foreign Exchange Market** (FOREX) denote a specific market, where the main traded goods are currencies. Traders involved in the market speculate on certain exchange pairs' volatility and bet on them in order to make a profit. The market itself determines price of every possible currency pair, thus making the acquisition of certain amount of currency possible through various ways – currency X could be traded through currencies Y, Z, etc., making the market far more speculative than others. By the volume of instruments traded, this decentralized market is by far the largest market in the world [19].

Being fully decentralized, meaning prices on this market are accepted worldwide, these data could be considered as a sufficient indicator of currency relative strength and, by that, an indicator of a particular government's and economic competitivness of a certain country. Taking this into account, we assume, that including such data inside our database would bring valuable information about the international competitivness of US economy towards other major economies and quantitative information about relations among major economies, which is most likely an important predictor for stock price movements.

By our focus on the US market, we chose to include currency pairs with the American dollar (**USD**) in all pairs. We chose to include so-called **major currency pairs**, with respect to their overall traded volume on a daily basis, according to reports published by the **Bank for International Settlements** and to market conventions [6, 19].

In particular, included pairs, together with the USD, are the Euro **(EUR/USD)**, Japanese yen **(JPY/USD)**, British pound sterling **(GBP/USD)**, Swiss franc **(USD/CHF)**, Australian dollar **(AUD/USD)**, Canadian dollar **(USD/CAD)**, and New Zealand dollar **(NZD/USD)**.

2.3.4 Other market indicators

Apart from the introduced indicators and the general assets' price movement data above, there is still a vast group of possibly valuable features, which do not fit into the above-defined features' groups.

In order to track the performance of the US economy more precisely, it is wise to include information regarding national debt as possible features. For this purpose, we included data about **US Treasury bonds**. Bonds denote a financial instrument mostly long-term, low-risk securities issued by the government of the USA, where an investor, in basic terms, lends money to the government in exchange for periodic fixed

payments. These bonds are further divided based on their maturity – the time within which issuer obliges itself to fully repay issued money.

Price movements of bonds with various maturities are considered to be a good indicator of the US economy, and thus of the stock market state [13]. In our database of possible features we included bonds with a maturity of 13 weeks, which belong to the group of short terms bonds, and bonds with 5 year long maturity. This kind of maturity is on the edge between long-term and medium-term. The choice reflects the need to track data regarding relatively very short horizon and a long one.

Apart from the bond-related data we considered obtaining data which are not directly financial – they are not any type of asset. We researched possibilities of tracking the sentiment of investors and of the market itself. We chose to include data of **Volatility Index** (VIX), which is a specific gauge tracking state of the US stock market in terms of volatility and the sentiment of investors.

It presumably brings valuable information, which might not be included in all of the metrics described, regarding the a priori mood of the market and its assets' risk. Its value ranges mostly up to 80, which was considered as crisis values in years 2020 and 2008, and above approximately 10, which is considered to be relatively stable market state. Value development between the January and September of 2025 is given in Figure 2.3.

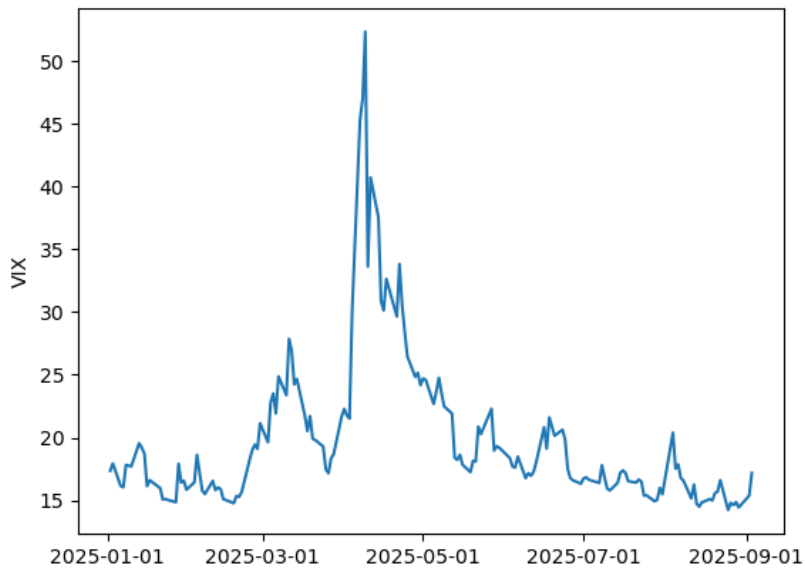

Figure 2.3: VIX value movement through the 2025.

It is surely interesting to address the sudden VIX surge during the April 2025. This sudden increase was basically day-to-day and denotes phase of repeated issuing and cancelling of international tarrifs. As we can see, this political move damaged investors' exposure to the US market then.

Chapter 3

Predictive analyses on IT segment

This chapter presents obtained modelling results. For each of the models used, in-depth parameter tuning and evaluation on both the training-validation and test datasets are performed.

3.1 Evaluation metrics

Throughout the thesis, particular metrics of goodness of regression are used. Suppose that on the day t we acquired closing price data Y_t; therefore, based on the methodology introduced earlier (Subchapter 2.2), we obtain a prediction for the day $t+1$ as $\hat{Y}_{t+1}$.

Firstly, in order to measure deviation from the actual Y_{t+1}, metric **absolute percentage error** (APE) is used. The percentage relativization was chosen in order to unify the scale among all the stocks, making the prediction error comparable across the stocks and more interpretable. The complete definition of the metric is given as

$$\mathrm{APE}(\hat{Y}_{t+1}, Y_{t+1}) = \frac{Y_{t+1} - \hat{Y}_{t+1}}{Y_{t+1}}.$$

Building upon the APE definition, the average value of the APE for individual stocks over the entire time interval considered is evaluated.

Apart from that, we introduce a baseline benchmark to measure the model's significance. According to Fama, the asset price developments correspond to the random walk [5]. In accordance with this, such naïve method can be introduced, that benchmark predicts the next-day price at the $t+1$ to be equal to the price on the day t.

With the benchmark method defined, we quantify performance as the proportion of predictions made by the model that are closer, in terms of APE, to the actual closing price than the predictions made by the naïve benchmark. From now on, such benchmarking will be referred as the **model's dominance**.

3.2 Linear regression

Although linear regression might seem rather simple, compared to other conventional ML frameworks it has been implemented in financial predictive modelling quite regularly. According to Zhao [25], linear regression is capable of reflecting the trend-based development of a certain stock up to a certain extent. Zhao suggests that linear regression alone might not reflect patterns in general for any stock.

On the other hand, Wang et al. [22] suggest that linear regression, with appropriate data preprocessing, is capable of performing short-term predictions up to a certain accuracy threshold.

Our next-day approach can be considered short-term; therefore, we decided to include this framework in the thesis. Apart from the conventional regression, we considered a possible feature selection optimization in each rolling window step and optimized the parameter p.

3.2.1 Possible feature selection

In order to analyze the possible contribution of omitting certain predictors, we implemented rolling forecasting with the selection of given k most relevant predictors with respect to the next-day closing price. Taking into account the potentially extensive computational time of such prediction, predictor selection based on F-statistics was used.

The statistics reflect the ratio of variance explained by a single predictor (with respect to the closing price) to the remaining unexplained variance. Predictors that achieve higher values of the statistics are considered strongly correlated with the target [9]. After such precomputation, the k best-performing predictors are kept in feature matrix.

However, such an approach tends not to perform significantly better than keeping all the predictors. Figure 3.1 shows a sample example of MAPE dependence on the number of selected best features for the stock Enphase Energy Inc. (ENPH).

It is possible to observe that omitting less correlated predictors causes and increase in MAPE. Up to a certain number k, the MAPE level tends to decrease. A further decrease in k results in a stable MAPE level (red dotted line), which represents the level of MAPE when no predictors are omitted.

Such behaviour was observed in the vast majority of stocks analyzed, with only slight (roughly $10^{-1} - 10^{-3}\%$) deviations below the benchmark line were observed. Furthermore, no feature selection in terms of linear regression is used.

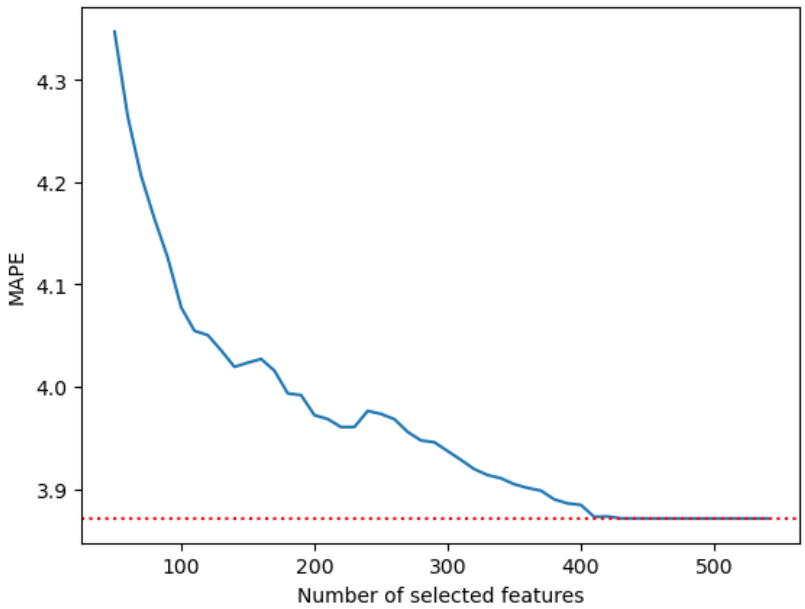

Figure 3.1: MAPE dependence on number of features selected (stock ENPH).

3.2.2 Optimization of p

The only a priori model parameter present in the linear regression is the number of observations included in the training (denoted as p). Analysis of the dependence of MAPE on the parameter p for the selected ENPH stock, is shown in Figure 3.2.

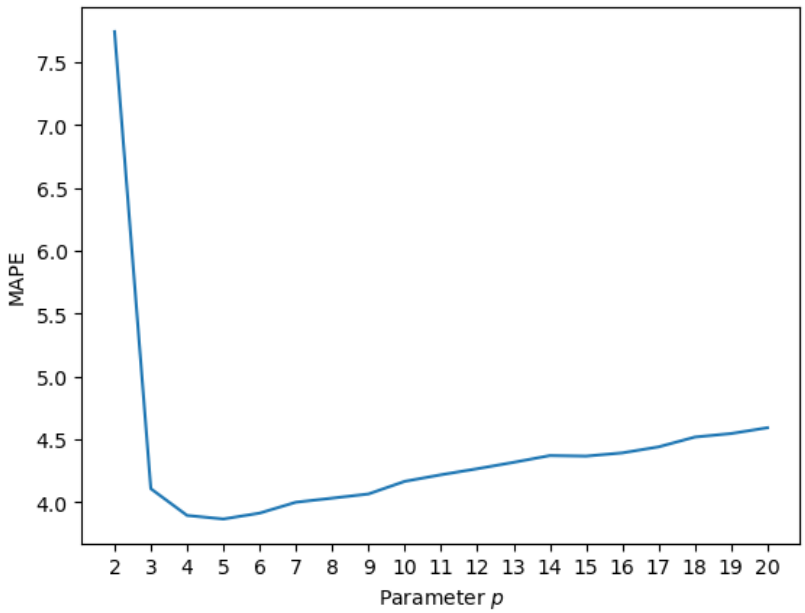

Figure 3.2: MAPE dependence on the rolling window training length parameter (stock ENPH).

In Figure 3.2, local minimum is observed at $p = 4$. In most of the included stocks

we observed a similar convex development of MAPE dependence on p. What is more, the minimum, in the vast majority of stocks, is observed in $p = 4$, rarely at $p = 5$. The value of $p = 4$ was chosen, based on the optimization on the training-validation interval.

Such a defined model achieved (on the training-validation interval) MAPE of **1.844 %** and dominance of **44.82 %** on average for all the stocks. In terms of interpretation, this means that linear regression is inferior, in terms of prediction accuracy, to the naïve benchmark on the given interval.

It is possible to analyze performance of the model for the given set of stocks. In Figure 3.3 is the overview of distributions of percentual deviations of predictions on the training-validation interval.

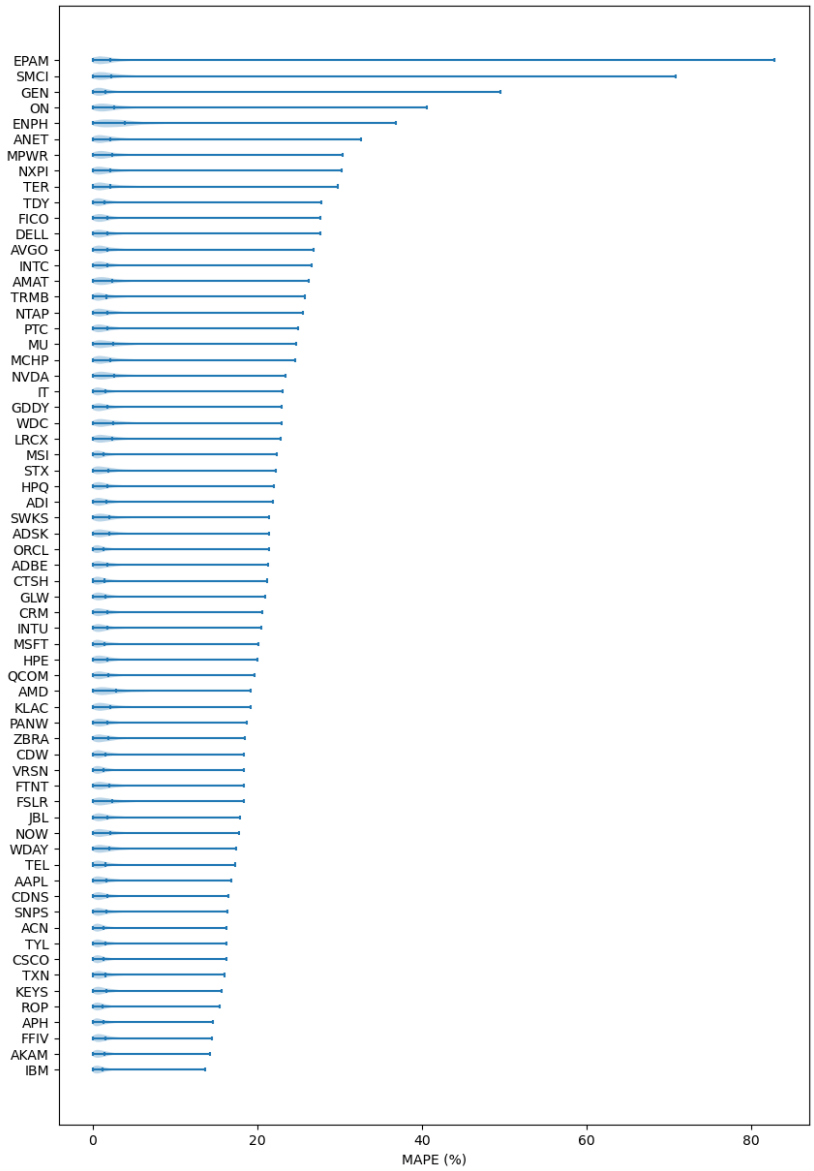

Figure 3.3: Errors in prediction for single stocks (linear regression).

In the vast majority of stocks, maximum error of 20% was observed; however even pathological predictions with abnormally high error was suspected. Even though, these cases were observed in minimal number of cases – average percentual error per stock did not exceed 4%.

3.3 Lasso regularization

Introducing regularization into the regression process might presumably improve the overall performance in terms of the introduced metrics, even though explicit feature selection has not proved to be sufficient. Apart from the linear regression framework it is necessary to further tune the regularization parameter λ, together with the length of the training set p.

Figure 3.4 visualizes the performance of MAPE depending on the chosen parameters λ and p for the ENPH stock.

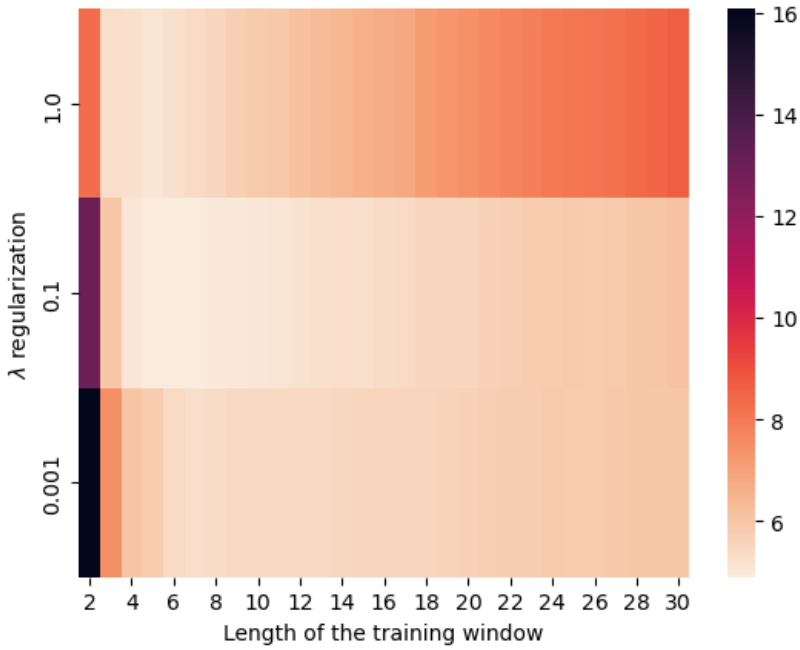

Figure 3.4: MAPE dependence on the rolling window training length parameter and regularization parameter (stock ENPH).

In terms of the parameter p, it is possible to observe that the lowest MAPE is achieved in the approximate interval $p \in [4, 8]$ for the chosen values of λ. Based on further evaluation of mean MAPE performance across all stocks, the lowest mean MAPE was achieved for $p = 4$.

Further, the interval of the optimal setting of λ for the given stocks is $\lambda \in [4, 5]$. Figure 3.5 depicts an overview of MAPE for the chosen ENPH stock. What needs

to be pointed out is the MAPE fluctuation on the given subinterval $[4.5, 4.7]$. The improvement remains negligible, reaching at most a magnitude of $10^{-3} - 10^{-2}\%$.

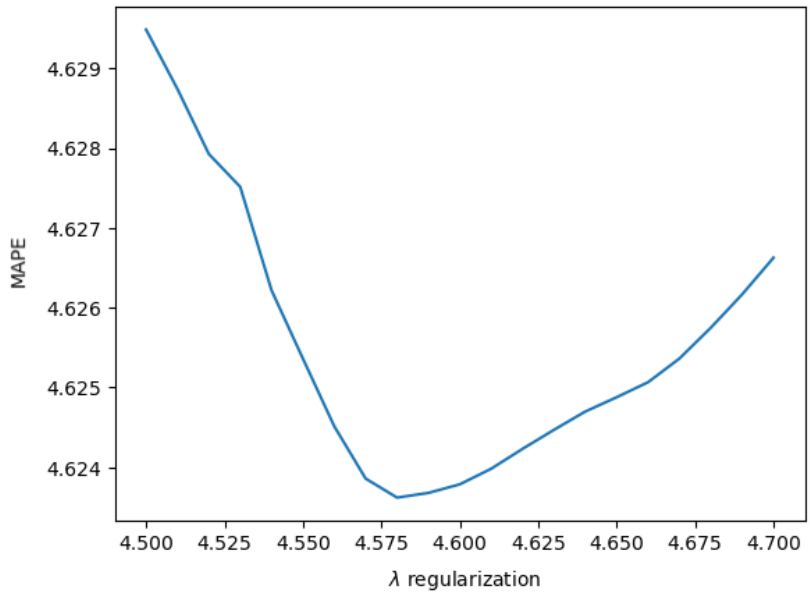

Figure 3.5: MAPE dependence on the rolling window training length parameter and regularization parameter in Lasso (stock ENPH).

With the optimized parameters, we decided to verify two approaches to setting up the model. First, we used optimized values of λ of each of the stocks. The resulting value is $\overline{\lambda} = 4.15$. We set up the Lasso model with the given $\overline{\lambda}$ (General model) and, in contrast, we verified performance of a model that identifies the predicted stocks and applies its particular optimized λ (Stock-wise model). The results of performance on the given training-validation interval are summarized in Table 3.1.

Model Type	**MAPE (%)**	**Dominance (%)**
General model	2.297	36.16
Stock-wise model	2.27	35.81

Table 3.1: MAPE and dominance values for different settings of the model (Lasso).

Generally speaking, compared to the linear regression, there is a marginal decline in terms of both metrics. What is interesting is the behaviour of both metrics. The General model has achieved comparatively worse MAPE; however, its dominance is slightly higher than that of the Stock-wise model.

Lastly, Figure 3.6 visualizes the performance of the optimized model for individual stocks. The visualization shows performance of the General model as there is no significant difference in the corresponding visualization of the Stock-wise model.

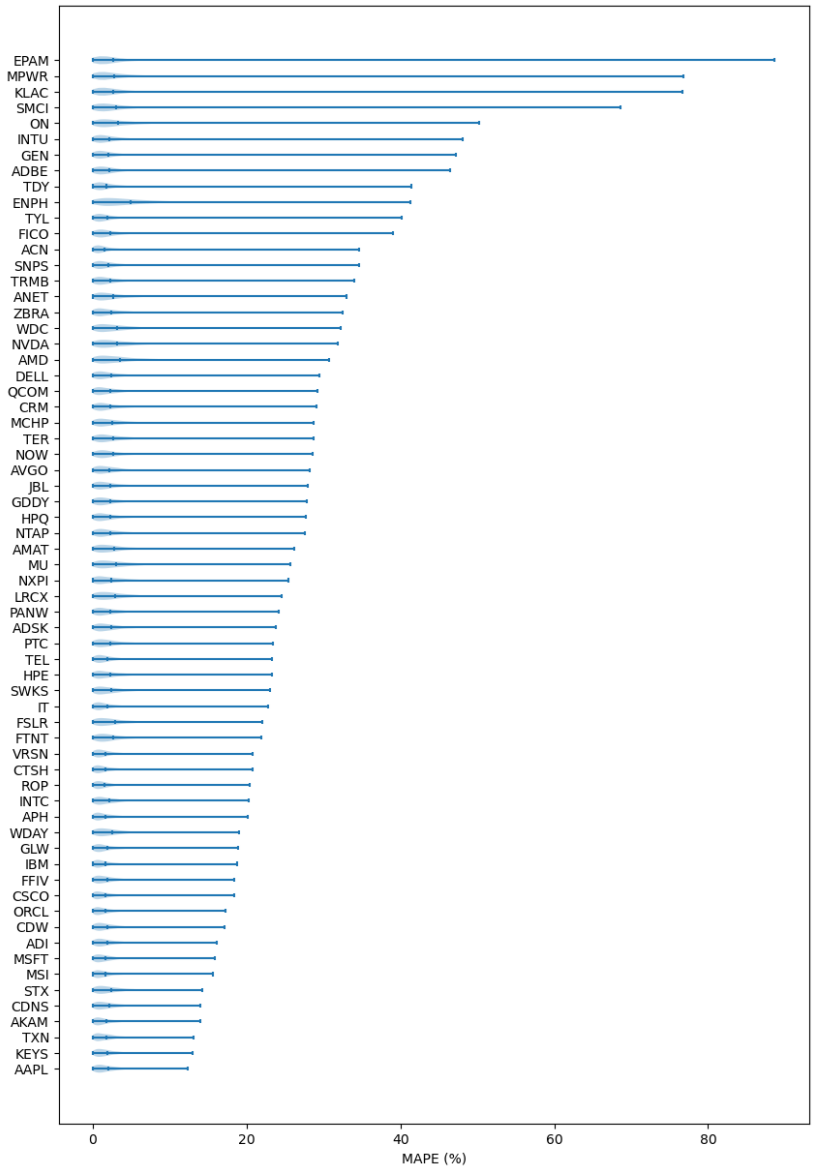

Figure 3.6: Errors in prediction for single stocks (General model Lasso).

It is possible to suspect a very similar distribution to the one in Figure 3.3, which visualizes distribution of error in linear regression. Even so, there are a few minor differences – the order of the stocks, in terms of the range of errors, is not preserved. For example, the lowest range in Lasso was achieved on AAPL stock (Apple Inc.); however, in linear regression, it was achieved for the IBM stock. This suggests that different ML algorithms most likely perform differently when predicting next-day price of single stocks – the stock having relatively the best prediction results under one algorithm does not necessarily perform best under another.

3.4 Ridge regularization

Similarly to Lasso, there are several properties of Ridge regularization, that might improve the performance of the regression, compared to the linear regression. The parameter tuning process in this case is analogous to the one used in Lasso (see Subchapter 3.3).

In Figure 3.7, the dependence of MAPE for the chosen ENPH stock on the selected regularization parameter λ and the training window length p is depicted.

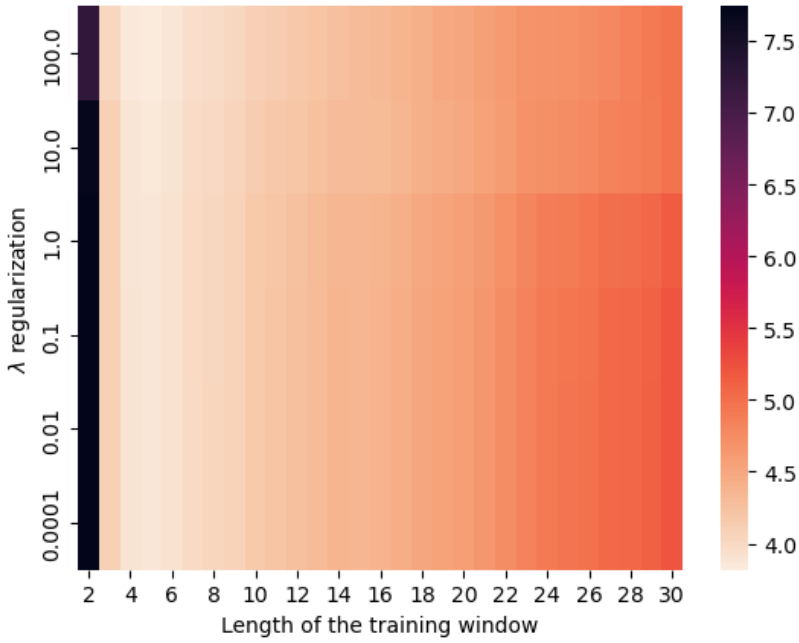

Figure 3.7: MAPE dependence on the rolling window training length parameter and regularization parameter in Ridge (stock ENPH).

Directly compared to the MAPE development in Figure 3.4, there is clear evidence of the optimal value of p being in the segment of $p \in [4, 6]$. The rise of error with the increasing value of p is much more visible and stable for various values of λ. With further experiments, value of $p = 4$ was chosen for later analyses. Analogous to the problem of choosing value of p in Lasso regression, a certain subinterval of near-optimal values was localized – the lowest value was chosen to save computational time.

As seen in Figure 3.8, dependence of MAPE on the value of the parameter λ tends to follow a convex course. The segment of optimal λ for the set of stocks lies in the interval $\lambda \in [170, 260]$.

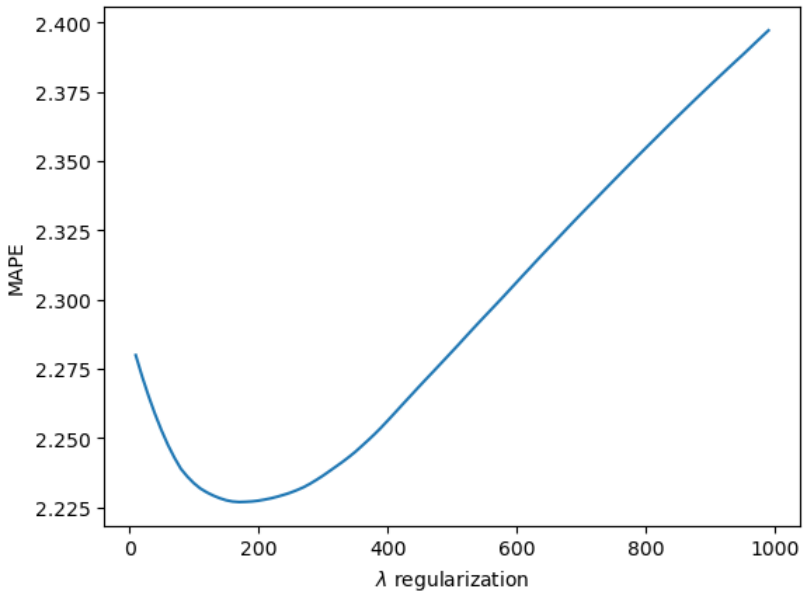

Figure 3.8: MAPE dependence on the rolling window training length parameter and regularization parameter in Ridge (stock ENPH).

Similar to the approach in Lasso regularization, using two model settings were chosen. The average value of optimal λ for the analysed stocks is $\overline{\lambda} = 211.85$ (General model). Similarly, a model using the optimal value of λ for each stock independently was also implemented (Stock-wise model). The final evaluation of these models is accumulated in Table 3.2.

Model Type	**MAPE (%)**	**Dominance (%)**
General model	1.791	45.08
Stock-wise model	1.79	45.15

Table 3.2: MAPE and dominance values for different settings of the model (Ridge).

Compared to the evaluation of linear regression, progress was achieved in both metrics progress. Ridge models have clearly better dominance and MAPE. Interestingly, compared to the case of Lasso, where counter-intuitive behaviour of MAPE and dominance was observed, the opposite behaviour is seen in Ridge models. General model tends to have higher MAPE and lower dominance, which is more intuitive.

Last but not least, an overview of error distributions for individual stocks in the models is depicted in Figure 3.9 (General model). Similar to the Lasso case, the distinctions between General and Stock-wise models are negligible.

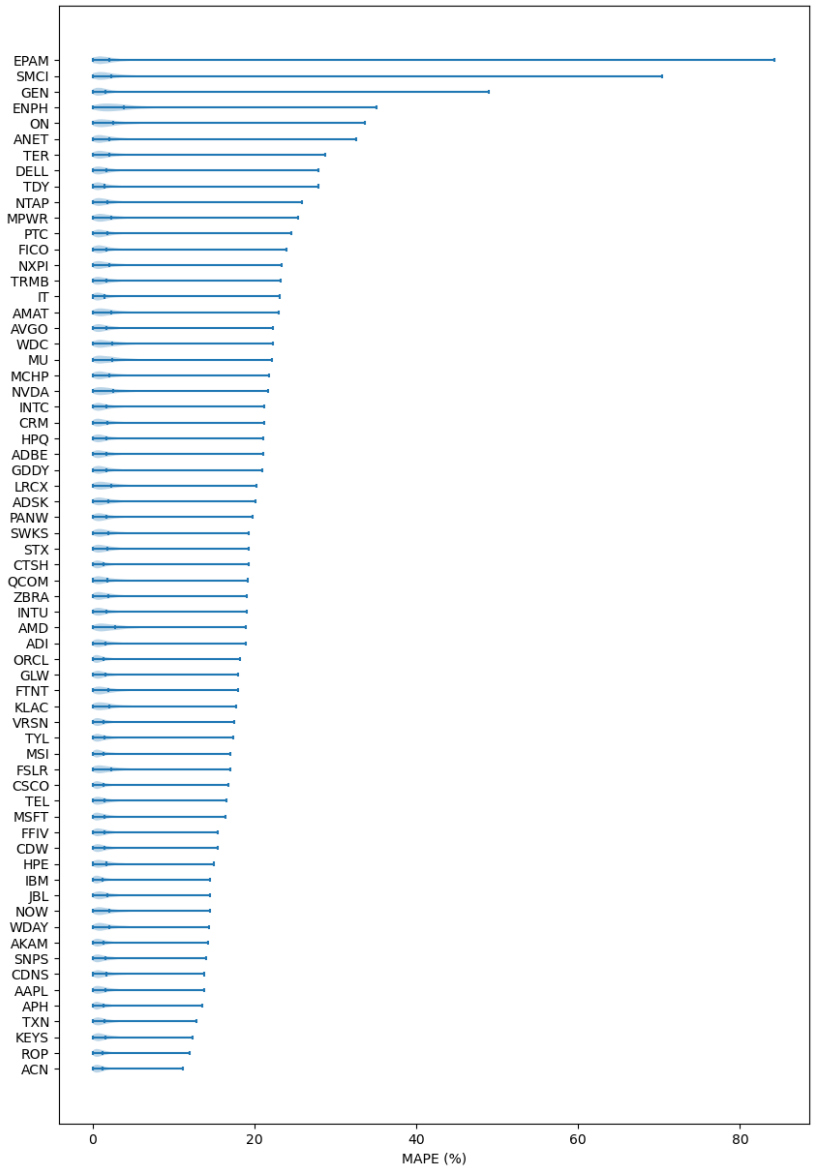

Figure 3.9: Errors in prediction for single stocks (General model Ridge).

Overall, there are only minor distinctions to the presented Figures 3.3 and 3.6, mainly in the order of stocks based on the error range. This again suggests that different ML frameworks tend to have varying performance across different stocks.

3.5 k-Nearest neighbours regression

Compared to the methods used earlier in this chapter, k-NN is a non-parametric method. The effect of this property on the predictive performance is definitely worth exploring. Although the framework seems simple and one might expect limited practical applicability, several studies demonstrate its potential and relevance in predicting stock prices.

Huang [8] included the k-NN algorithm in the stack of algorithms applied. Even though k-NN did not prove to be the best-performing among the algorithms used, Huang states that k-NN performs better in terms of the employed evaluation metric than more complex support vector regression framework.

Further, Yin et al. [24] showed that the k-NN algorithm tends to perform well, as their analysis of certain US stocks demonstrated that the deviation of predictions is relatively low, compared to other frameworks even for longer forecast periods.

In the case of this algorithm, it is necessary to analyze the combined effect of choosing both parameters k (number of neighbours) and the length of the training window p. Figure 3.10 shows the effect of dependence of MAPE on given values of parameters. The visualization is based on the averaged results of four given stocks: Roper Technologies Inc. (ROP), Enphase Energy Inc. (ENPH), Cadence Design Systems Inc. (CDNS), and EPAM Systems Inc. (EPAM).

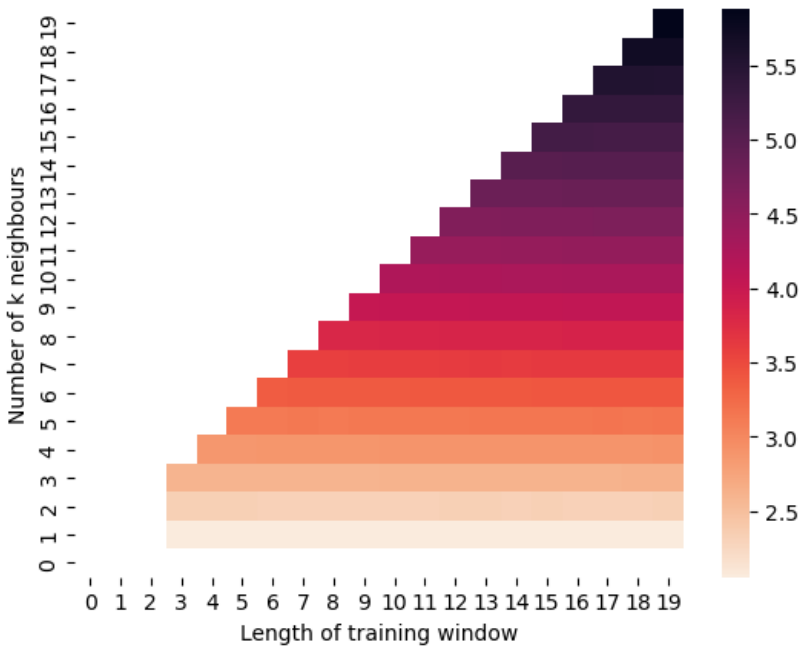

Figure 3.10: Dependence of k and p parameters on MAPE of next-day predictions (k-NN; average among chosen stocks).

There is a clear trend showing that the most influential parameters is k, number of neighbours. The most interesting observation is the almost constant MAPE across all the considered values of p. This appears to be highly counter-intuitive quality; however, further analyses have shown that it could be explained as a form of pathological behaviour of k-NN algorithm.

Across all stocks k-NN tends to reach its optimal setting when $k = 1$. In all rolling window steps across all the stocks, the closest 'neighbour' to the prediction input is the observation from the most recent previous day. As k-NN has optimal setting in $k = 1$

regardless of p, k-NN algorithm effectively performs as a form of the proposed naïve benchmark. Consequently, k-NN cannot outperform it. Such setting $(k = 1, p = 4)$ has achieved average MAPE per stock **1.69 %**.

To complete the analysis, Figure 3.11 depicts the distribution of errors across stocks.

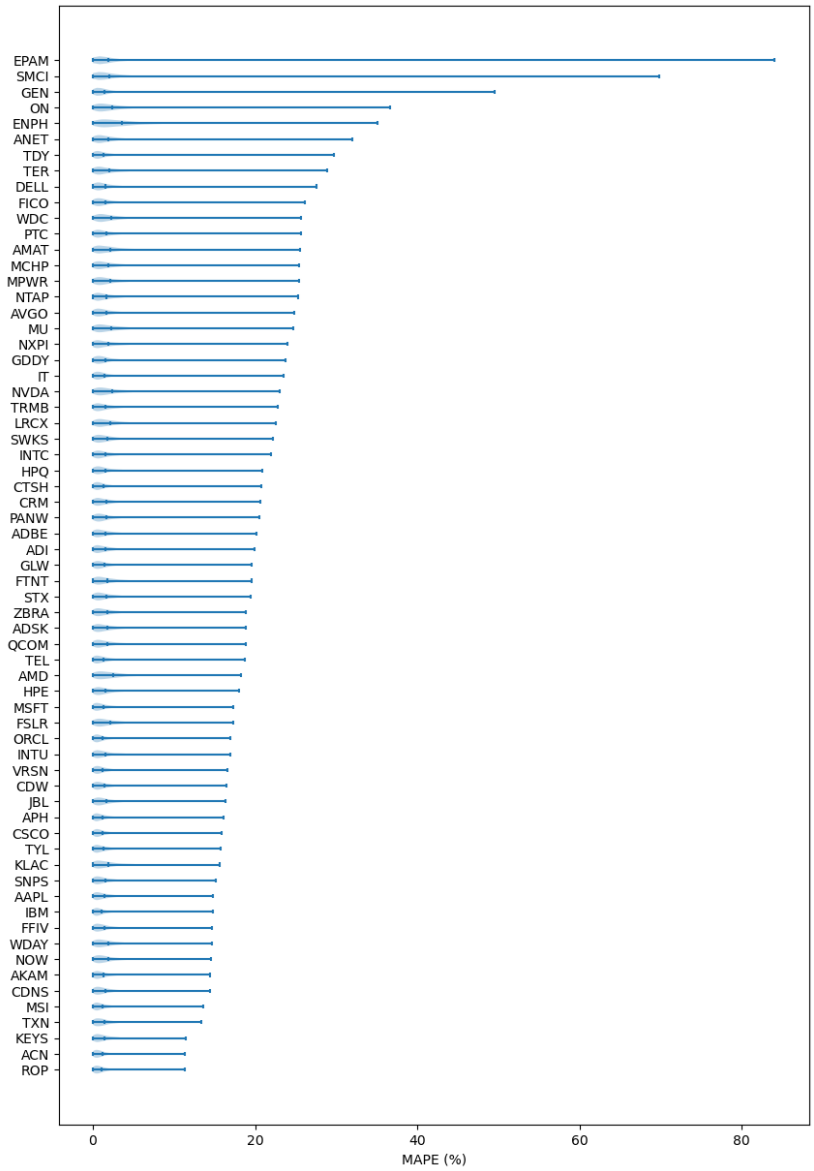

Figure 3.11: Errors in prediction for single stocks (k-NN).

Again, the same property of model applies here as well. The individual error distributions tend to be very similar; however, the order of stocks based on range of the error is slightly shuffled. Interestingly, the order of stocks is very similar to the one in Figure 3.9 (Ridge). This may root from the fact that these models are very close to each other in terms of MAPE per stock.

3.6 Random forest regression

The Random forest regression framework is a frequently adopted algorithm in the finance area. There are studies employing it to predict the next-day price developments. Khaidem et al. [11] implemented Random Forest in its classifier form to predict direction of change of the next-day price. Using specific feature engineering, the authors suggest that this framework is capable of outperforming random guessing.

However, there are few papers adopting the regression form of Random Forest for stock price prediction. Krauss et al. [12] proposed an ensemble implementation of Random Forest in combination with other algorithms, but reported performance only on the S&P 500 index.

In the analyses conducted in this thesis, Random Forest Algorithm (RFA) aligns with many phenomena observed earlier. In Figure 3.12, the dependence of MAPE, for a single stock, on the parameter p – the length of the training window – is shown.

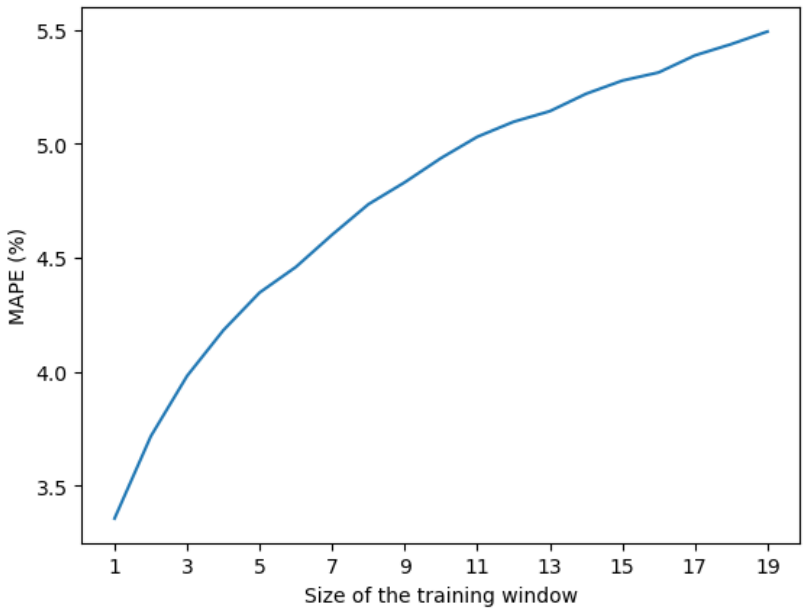

Figure 3.12: Dependence of MAPE on the length of the training window p (stock ENPH, Random Forest).

A distinct feature, similar to the one explored in the Subchapter 3.5, can be observed. For all individual stocks in the datasets, RFA tends to perform the best in terms of MAPE when $p = 1$. Effectively, this means that training phase consists of only the most recent known data entry – the previous trading day. Consequently, RFA tends to fall into explained pathological case, equivalent to the proposed naïve benchmark.

Moreover, tuning the available parameters (e.g., number of decision trees in the forest, maximum depth of single trees, etc.) does not change the course of parameter

optimization. In all analyses and across all analyzed stocks, the model setting $p = 1$ performs best. In addition, for settings $p > 1$, there is no improvement in dominance compared to the approaches presented above using other algorithms. Figure 3.12 shows dependence of model dominance on the parameter p – the best setting in terms of the dominance is $p = 2$.

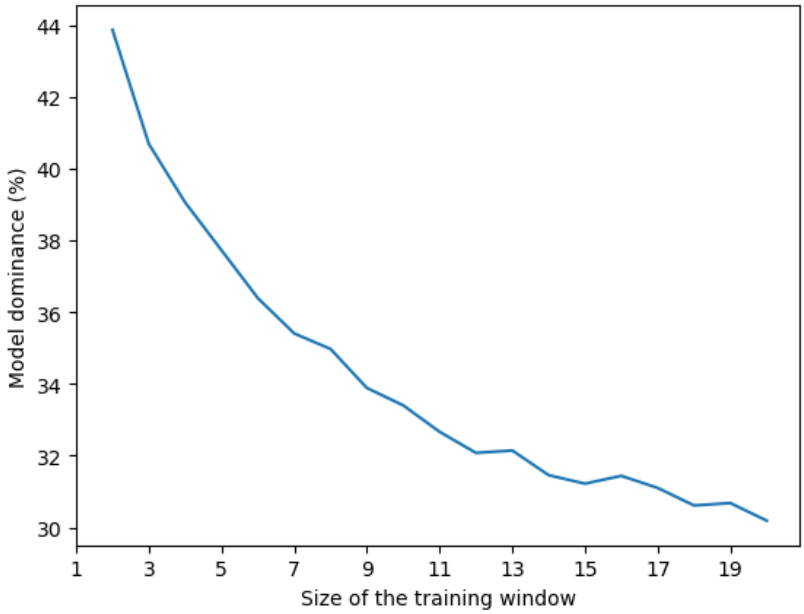

Figure 3.13: Dependence of model dominance on training window length (stock ENPH).

According to the analyses, this means that evaluation of MAPE is the same as in the k-NN framework. The value of average MAPE per stock (in the optimal model) is **1.69 %**.

3.7 ARIMA regression

As mentioned in Subchapter 1.7, ARIMA framework is capable of capturing time dependencies by default. This can be beneficial for predictive analysis in this context. In order to tune parameters p, d, q of the ARIMA model, an approach based on the principles of grid search was used.

For each particular rolling window, initial state is set as $p = 0, d = 0, q = 0$. Incrementally, algorithm evaluates neighboring settings, such as $p+1, q+1, d+1$. For each of the combinations (up to the defined upper limit of each parameter), model is fitted according to the maximum likelihood estimation (according to the Equation (1.16)). For each fitting, Bayesian information criterion is evaluated and finally, based on this metric, the best-performing setting is selected [21].

This method is somewhat naïve optimizer and, in terms of time complexity, computationally cumbersome. However, for the purposes of this thesis, this approach is suitable.

In this case, there is no need to present any dependence of MAPE or dominance on the chosen model parameters. For each rolling window and for each stock within it, parameters are chosen ad hoc.

On the training-validation interval, this optimized model achieved an average MAPE per stock **1.897 %** and dominance of **58.25 %**. Although the model was not able to outperform naïve benchmark or even Ridge in terms of MAPE, it performed best overall in terms of model dominance. Such result is particularly interesting, as a comparatively lower MAPE does not necessarily imply higher dominance.

3.8 Test evaluation

In the previous subchapters, the process of parameter tuning was outlined in detail and, based on the introduced metrics, evaluated in order to optimize model performance. As introduced earlier, the same rolling-window-based prediction approach will be used on the test interval as well (see Subchapter 2.2).

The evaluation of all implemented algorithms on the test interval is shown in Table 3.4. For completeness and easier interpretation purposes, Table 3.3 also summarizes the results obtained on training-validation interval as well. Values of model dominance for k-NN and RFA are blank as they have no interpretative value – models are equivalent to the naïve benchmark.

Model Type	**MAPE (%)**	**Model Dominance (%)**
ARIMA	1.897	58.25
Ridge (Stock-wise model)	1.79	45.15
Ridge (General model)	1.791	45.08
Linear regression	1.844	44.82
Lasso (General model)	2.297	36.16
Lasso (Stock-wise model)	2.27	35.81
Random Forest	1.69	
k-Nearest neighbours	1.69	

Table 3.3: Evaluation of tuned models on the training-validation interval.

Model Type	**MAPE (%)**	**Model Dominance (%)**
ARIMA	1.923	56.68
Ridge (Stock-wise model)	1.794	45.94
Ridge (General model)	1.798	45.88
Linear regression	1.848	43.47
Lasso (General model)	2.24	38.83
Lasso (Stock-wise model)	2.31	37.1
Random Forest	1.72	
k-Nearest neighbours	1.72	

Table 3.4: Evaluation of tuned models on the test interval.

As seen in Tables 3.3 and 3.4, in both cases the metrics tend to reach similar values. This suggests that the models have not suffered from overfitting. Apart from that, it is interesting to observe similar values of both metrics on comparatively wide intervals. This may indicate that models are relatively stable.

Chapter 4

Next-day optimization of portfolio based on model's prediction

This chapter will deal with portfolio management using the obtained predictions.

4.1 Markowitz mean-variance portfolio

4.2 Portfolio evaluation metrics

4.3 Evaluation on the training-validation interval

4.4 Evaluation on the test interval

Conclusion

This section will be dedicated to concluding our results, accumulating them into a concise form, evaluating successfulness of completing given goals and suggesting future tweaks in order to make thesis' idea more robust and practice-friendly.

Bibliography

[1] Ran Aroussi. yfinance. https://github.com/ranaroussi/yfinance. Accessed: 2025-09-29.

[2] Dave Bergmann. What is machine learning? https://www.ibm.com/think/topics/machine-learning. Accessed: 2025-02-01.

[3] Nikita Chhabra et al. 2024 dividend trends. https://elements.visualcapitalist.com/wp-content/uploads/2024/09/1726513220934.pdf. Accessed: 2025-12-04.

[4] Francis X. Diebold. *Elements of Forecasting*. Thomson South-Western, 2006.

[5] Eugene F. Fama. Efficient capital markets: A review of theory and empirical work. *Journal of Finance*, 25(2):383–417, 1970.

[6] Bank for International Settlements – Monetary and Economic Department. OTC foreign exchange turnover in April 2025. *Triennial Central Bank Survey*, 2025.

[7] Jakub Grudniewicz and Rafal Ślepaczuk. Application of machine learning in algorithmic investment strategies on global stock markets. *Research in International Business and Finance*, 66, 2023.

[8] Yuping Huang. Research on the Google Stock Price Prediction Based on SVR, Random Forest, and KNN Models. *Highlights in Business, Economics and Management*, 24, 2024.

[9] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning*. Springer, 2023.

[10] Will Kenton. S&P 500 Index: What It's for and Why It's Important in Investing. https://www.investopedia.com/terms/s/sp500.asp?utm_source=chatgpt.com. Accessed: 2025-12-03.

[11] Luckyson Khaidem, Snehanshu Saha, and Sudeepa R. Dey. Predicting the direction of stock market prices using random forest. *Expert Systems with Applications*, 83:86–99, 2017.

[12] Christopher Krauss, Xaun Do, and Nicolas Huck. Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. *European Journal of Operational Research*, 259, 2017.

[13] Ronald R. Kumar, Peter J. Stauvermann, and Hang Vu. The Relationship between Yield Curve and Economic Activity: An Analysis of G7 Countries. *Journal of Risk and Financial Management*, 62, 2021.

[14] Pablo Mantilla and Sebastián Dormido-Canto. A novel feature engineering approach for high-frequency financial data. *Engineering Applications of Artificial Intelligence*, 125, 2023.

[15] Akito Matsumoto, Andrea Pescatori, and Xin Wang. Commodity prices and global economic activity. *Japan and the World Economy*, 66, 2023.

[16] Charlotte Morabito. The S&P 500 is more concentrated with AI than ever. Here's how to manage your risk. `https://www.cnbc.com/2025/10/22/your-portfolio-may-be-more-tech-heavy-than-you-think.html#:~:text=The%20top%20five%20companies%20in,relying%20on%20so%20few%20companies.` Accessed: 2025-12-03.

[17] Jasper Osita. Why Gold Remains the Ultimate Security in a Shifting World. `https://acy.com/en/market-news/education/why-gold-remains-ultimate-security-shifting-world-j-o-04222025-115842/`. Accessed: 2025-11-27.

[18] Sakina and Muhammad Khan. Stock Price Prediction using Neural Networks. *Pakistan Journal of Law, Analysis and Wisdom*, 3, 2024.

[19] ScienceDirect. Foreign Exchange Market, 2025. In: ScienceDirect. Available at: `https://www.sciencedirect.com/topics/economics-econometrics-and-finance/foreign-exchange-market#:~:text=The%20foreign%20exchange%20market%2C%20or%20forex%20market%2C,hours%20a%20day%2C%20five%20days%20a%20week.`. [accessed 2025-11-28]. AI generated definition based on: Strategies of Banks and Other Financial Institutions, 2014, Available at: `https://www.sciencedirect.com/science/chapter/monograph/abs/pii/B9780124169975000051`.

[20] Steven S. Skiena. *The Data Science Design Manual*. Springer, 2017.

[21] Taylor G. Smith. pmdarima. `https://alkaline-ml.com/pmdarima/`. Accessed: 2026-02-23.

[22] Qi Wang, Chang Xu, and Tieyan Zhou. Stock price prediction based on multiple linear regression. *BCP Business & Management*, 36:48–54, 2023.

[23] Yahoo. Yahoo Finance. `https://finance.yahoo.com/`. Accessed: 2025-09-29.

[24] Leying Yin, Xinyu Zhang, and Xiwen Xiong. Application and Effectiveness of KNN Model in Stock Market Prediction. In *Proceedings of the 3rd International Conference on Financial Technology and Business Analysis*, pages 129–133, 2025.

[25] Yuqing Zhao. Stock Price Prediction Based on Linear Regression. In *Proceedings of the 4th International Conference on Computing Innovation and Applied Physics*, pages 85–90. EAI, 2025.

Appendix A: Source code of the model

Appendix attached to this thesis contains all the source codes used in analyses introduces in the thesis. All the source codes and results are freely available as a Git repository at `https://github.com/sugy-w/Bachelor-Thesis`.

Please note, that all the contents of this repository is being shared only for academic purposes and its modifying, redistribution or commercial use is strictly prohibited by the included licenses.